

Dokumentation zu SMART Grid Austria & Speicher AI

Sprache: DE

1 AUFGABENSTELLUNG

Ziel dieses Projektes war die Entwicklung einer interaktiven Softwareanwendung zur Analyse von Netzkapazitäten, der Berechnung von Vollbenutzungsstunden (VBS) für erneuerbare Energien sowie der KI-gestützten Optimierung einer Batterie-Pufferallokation (BESS).

Die Kernidee und Hauptaufgabe des Projektes bestand in einer zweistufigen Daten-Architektur (Data Pipeline):

1. **Datenbeschaffung (ETL-Prozess):** Reale Auslastungsdaten werden von der offiziellen API des Netzbetreibers online abgerufen, um die exakte, aktuell freie Leistung zu ermitteln. Parallel dazu werden historische Klimadaten (via Open-Meteo API) für die VBS-Berechnung bezogen.
2. **Daten-Caching & Dashboarding:** Um eine hohe Performance des interaktiven Dashboards zu garantieren und API-Rate-Limits zu vermeiden, werden diese fusionierten Daten in einem lokalen Cache (`substations_climate_base.csv`) zwischengespeichert. Das Dashboard greift auf diesen performanten Daten-Schnitt zu.

Die entwickelte Lösung sollte über ein interaktives Dashboard zugänglich gemacht und für eine plattformunabhängige Ausführung in Docker containerisiert werden.

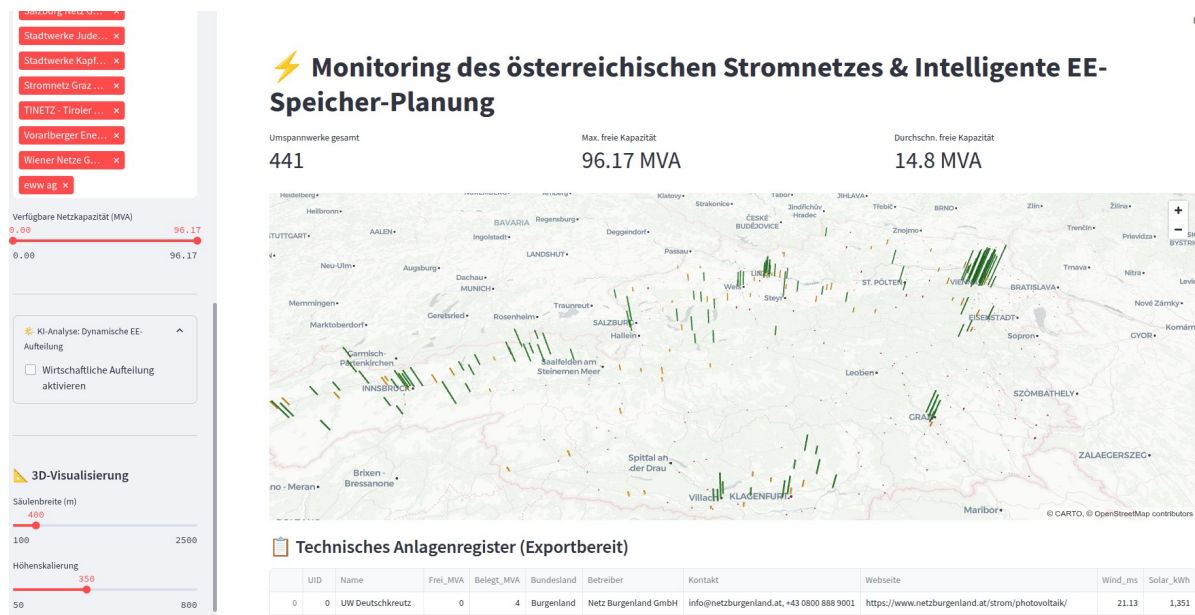


Abbildung 1 Benutzeroberfläche des SMART Grid Dashboards mit interaktiver Karte

Wie in Abbildung 1 dargestellt, ermöglicht das Dashboard die dynamische Eingabe von Parametern und visualisiert die Ergebnisse basierend auf dem generierten Datenschnitt in Echtzeit auf der Karte.

Tabelle 1: Übersicht der Kernkomponenten und Datenquellen

Komponente	Technologie	Zweck
Netzdaten (Freie Leistung)	Offizielle API	Pre-fetched Online-Daten (Aktueller Live-Schnitt zum Zeitpunkt des Skriptaufrufs, lokal im CSV-Cache für maximale Performance gespeichert)
Klimadaten (VBS-Basis)	Open-Meteo Archive API	Statisch / Historisch (Mittelwerte des Jahres 2024, da sich Klimadaten nicht kurzfristig ändern)
Visualisierung	Streamlit & pydeck (3D-Mapping)	Interaktive Darstellung im Browser
Deployment	Docker & Git	Isolierte Ausführungsumgebung

Semester/Durchführungszeitraum S2026

Wie aus Tabelle 1 ersichtlich, kombiniert das System aktuelle Netzdaten-Schnittmengen mit historischen Klimawerten. Die richtige Zitierweise von externen Tools und Quellen wurde im Literaturverzeichnis angewandt [1] [2] [3].

2 HERANGEHENSWEISE / HOW-TO BESCHREIBUNG

Die Implementierung erfolgte modular in Python durch eine Trennung von Backend-Pipeline und Frontend-Anwendung:

- **Data Pipeline (`fetch_climate_data.py`):** Dieses Skript fungiert als Daten-Crawler. Es sendet automatisierte Web-Requests an die Plattform der Netzbetreiber, parst die Antwort, holt standortspezifische Wetterdaten und speichert den bereinigten Datensatz als CSV-Cache ab.
- **Analytisches Frontend (`app_climate_analysis.py`):** Die Streamlit-Applikation lädt den vorbereiteten CSV-Cache. Dies ermöglicht blitzschnelle Filterungen und die sofortige, verzögerungsfreie Berechnung der KI-gestützten BESS-Allokation.

Um das System hardwareunabhängig zu machen, wurde ein `Dockerfile` geschrieben. Dies erlaubt es dem Nutzer, die Anwendung mit minimalem Aufwand in einer isolierten Umgebung zu starten.

3 AUFGETRETENE PROBLEME

Während der Entwicklungs- und Bereitstellungsphase traten folgende technische Herausforderungen auf:

1. **API-Limitierungen & Ladezeiten:** Initiale Versuche, die Netz- und Wetter-APIs bei jedem Klick im Dashboard live abzufragen, führten zu inakzeptablen Ladezeiten und drohenden IP-Sperren. Die Lösung war die Implementierung der beschriebenen Caching-Architektur (CSV-Pre-fetching).
2. **Umgebungsabhängigkeiten:** Lokale Ausführungsversuche scheiterten zunächst an fehlenden Modulen (z.B. `streamlit`). Dieses Problem wurde durch die saubere Containerisierung via Docker gelöst.
3. **GitHub Authentifizierung:** Beim Push des finalen Codes in das Repository trat ein Error 403 auf, der durch die Nutzung eines dedizierten Classic Tokens für die CLI-Authentifizierung behoben wurde.

4 ERGEBNISSE

Das Resultat ist ein voll funktionsfähiges, Docker-basiertes System zur Analyse von Smart-Grid-Daten. Das System automatisiert den Abruf von Netz- und Klimadaten über eine Pipeline, speichert diese performant ab und stellt die optimale Speicherladestrategie auf einer interaktiven Karte flüssig dar.

5 BEWERTUNG DER ERGEBNISSE

Das System erfüllt alle funktionalen Anforderungen und läuft dank der Caching-Architektur und Docker äußerst stabil und ressourcenschonend. **Shortcomings (Datenaktualität):** Ein designbedingtes Shortcoming dieser Caching-Architektur ist, dass das Dashboard Änderungen im physischen Stromnetz nicht in der absoluten Sekunde ihres Auftretens widerspiegelt. Wenn sich die freie Kapazität eines Umspannwerks ändert, muss das Skript `fetch_climate_data.py` erneut ausgeführt werden, um den Daten-Schnitt zu aktualisieren. Für strategische Langzeitplanungen von BESS-Systemen ist dieses Intervall-Update jedoch branchenüblich und vollkommen ausreichend.

6 AUSBLICK

Um das Thema in Zukunft weiterzuführen, bestehen folgende Ideen:

- **Automatisierung der Pipeline:** Einrichtung eines Cronjobs (oder Celery-Tasks) innerhalb des Docker-Containers, der das Fetch-Skript automatisch z.B. alle 15 Minuten im Hintergrund ausführt, um den Cache ohne manuelles Eingreifen aktuell zu halten.
- **Prädiktive Analysen:** Einsatz von Machine-Learning-Modellen (z.B. LSTMs), um die zukünftige freie Netzkapazität basierend auf den historisch gesammelten Cache-Daten vorherzusagen.

7 LITERATUR

- [1] Streamlit Inc. "Streamlit Documentation". Online verfügbar: <https://docs.streamlit.io/> (abgerufen am 20.06.2024)
- [2] Docker Inc. "Docker Documentation". Online verfügbar: <https://docs.docker.com/> (abgerufen am 20.06.2024)
- [3] Open-Meteo. "Open-Meteo Historical Weather API". Online verfügbar: <https://open-meteo.com/en/docs/historical-weather-api> (abgerufen am 20.06.2024)